

# Notes on the Identity Axis

Unifying A1, A2, and B1 Notes around KYC-Bound Identity M

Perry Kundert

2026-06-04



A Note’s authority axis is the KYC-bound Identity point  $M$ , never the Ethereum EOA or contract address: a Note *names* the issuing and receiving Identity, and accounts are interchangeable envelopes that need only *match* the expected  $M$ . This document shows that making that axis explicit – by adding a **registry-Identity Merkle accumulator** (root on chain, a fresh root committed *before* a newly issued  $M$  can be used) – collapses the entire Notes system onto a **single gadget**: prove a re-encryption of a *registered* Identity (a membership in the accumulator) under a target key. The three flavours are one gadget, instantiated by *who encrypts, when, and under what key*: **A2** (the issuer encrypts its Identity under the recipient identity point, at mint), **A1** (a public issuer, in the clear), and **B1** (the depositor encrypts *its own* Identity under the public issuer’s key, at spend). The A2 and B1 bindings are exact duals, and **both are implemented and on-chain-verified** (`IdentityRegistry.verifyDepositCoupling`, `verifyDepositorBinding`). The B1 dual answers the open question directly: a public bearer issuer gains *collusion-resistant* knowledge of the unencrypted Identity of every depositor of its notes – unknown at mint – while Mallory learns nothing. (Notes, Unilateral A2, Decryptability)

## Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>The Authority Axis Is Identity, Not the Account</b>     | <b>3</b>  |
| <b>2</b> | <b>The Registry-Identity Accumulator</b>                   | <b>4</b>  |
| <b>3</b> | <b>The One Gadget</b>                                      | <b>5</b>  |
| <b>4</b> | <b>A2 – The Recipient Names the Private Issuer (recap)</b> | <b>6</b>  |
| <b>5</b> | <b>B1 – The Public Issuer Names the Bearer Depositor</b>   | <b>7</b>  |
| 5.1      | The construction . . . . .                                 | 7         |
| 5.2      | The issuer-unilateral receipt . . . . .                    | 7         |
| 5.3      | Why it is collusion-resistant and private . . . . .        | 7         |
| <b>6</b> | <b>The Unified Spend Relation</b>                          | <b>8</b>  |
| <b>7</b> | <b>What This Simplifies and Strengthens</b>                | <b>9</b>  |
| <b>8</b> | <b>Key-Management Semantics</b>                            | <b>10</b> |
| <b>9</b> | <b>Implementation Status</b>                               | <b>11</b> |
| 9.1      | Migration . . . . .                                        | 11        |
| 9.2      | Cross-references . . . . .                                 | 11        |

**STATUS (2026-06-10): historical – superseded.** This design memo’s content is absorbed: the identity-axis inversion and one-gadget model live in *alberta-buck-notes* ("Mutual Decryptability", "Long-Term Holding Safety"); the per-flavour mechanics and executable walkthroughs in *alberta-buck-notes-flow* ("The Identity-M Spend Path", the "in Code" sections); the formal statements in *alberta-buck-proofs* (Part IV, Theorems 7-12). The implementation-status section below reflects June 2026 and is no longer maintained.

## 1 The Authority Axis Is Identity, Not the Account

The shipped Notes design is *account-centric*: the A-spend pins the note to a specific mint-time account via a Chaum-Pedersen equality that forces  $pk_{\text{rec}} = pk_{\text{dep}}$  (so key loss strands the note), and the B-spend carries *no* in-circuit Identity at all (the depositor-naming is an optional on-chain step, "sound for an honest depositor" only). Two divergent spend circuits, two binding mechanisms, and an account that is load-bearing.

But a Note is not a transfer between accounts; it is a claim by one *Identity* on the commons, redeemable by another *Identity*. The unilateral-A2 result (*alberta-buck-notes-unilateral*) made this concrete: encrypt under the recipient’s Identity *point*  $M_{\text{rec}}$  and the note is redeemable by *any* Fountain account bound to  $m_{\text{rec}}$ . Pushed to its conclusion, the account drops out of the design and one structure remains.

## 2 The Registry-Identity Accumulator

**Definition 1.** *The identity accumulator is a Poseidon Merkle tree whose leaves are registered Identity points,  $\ell(M) = \text{Poseidon}(M.x, M.y)$ , maintained by the **IdentityRegistry** on every **register** / **bindContract**. Its root  $\text{rt}_{\text{id}}$  is published on chain; the full tree lives off chain (the wallet's Holochain layer – no size limit). Membership of  $M$  in  $\text{rt}_{\text{id}}$  is the statement " $M$  is a genuine, KYC-registered Identity." (`alberta_buck.wallet.unilateral_a2.IdentityTree`.)*

**Commit-before-use.** A newly issued  $M$  must be folded into the on-chain root *before* it is used as a Note counterparty: the issuance is final only once  $\text{rt}_{\text{id}}$  reflects it. This is the standard incremental-accumulator discipline (Tornado / Semaphore): membership proofs reference a finalised root, and an unregistered  $M$  is structurally unusable. Maintenance is an on-chain incremental Poseidon root (gas per registration) or an off-chain-computed root proven by a small SNARK; the tree body stays off chain.

### 3 The One Gadget

Every binding in the system is one relation, parameterised:

**Definition 2.** (Registered re-encryption.) *Given a target key  $K$  and a published ciphertext  $E = (R, C)$ , prove knowledge of a scalar  $m$  (and randomness  $r$ ) such that*

$$R = rG, \quad C = mG + rK, \quad \ell(mG) \in \mathbf{rt}_{\text{id}}$$

*i.e.  $E$  encrypts, under  $K$ , a registered Identity  $M = mG$  the prover controls – revealing neither  $m$  nor  $M$ .*

The EC half  $(R, C)$  is a cheap Okamoto sigma over EIP-196; the membership half  $(\ell(mG) \in \mathbf{rt}_{\text{id}})$  is a *native* Poseidon-Merkle proof. Three substitutions of  $(K, \text{ who proves, when})$  give the three flavours.

| Flavour   | $K$ (target key)                          | Who proves | When  | Effect                                                    |
|-----------|-------------------------------------------|------------|-------|-----------------------------------------------------------|
| <b>A2</b> | recipient identity point $M_{\text{rec}}$ | issuer     | mint  | recipient decrypts -> names the (private) issuer          |
| <b>A1</b> | – (issuer $M_I$ in the clear + signature) | issuer     | mint  | issuer named directly; recipient proves identity at spend |
| <b>B1</b> | public issuer key $pk_{\text{iss}}$       | depositor  | spend | issuer decrypts the event -> names the (bearer) depositor |

A2 and B1 are *duals*: the same gadget, run by opposite parties, toward opposite keys. Both are implemented and on-chain-verified.

## 4 A2 – The Recipient Names the Private Issuer (recap)

The issuer encrypts its own registered Identity under the recipient identity point,  $eIss = (r'G, M_I + r'M_{\text{rec}})$ , and the depositor proves eligibility with the deposit coupling (`verifyDepositCoupling`), all Identities hidden:

$$\begin{aligned} \text{(E4)} \quad & pk_{\text{dep}} = sk_{\text{dep}}G; \\ \text{(E2)} \quad & C_d = m_{\text{rec}}G + sk_{\text{dep}}R_d; \\ \text{(E3)} \quad & C_e - P_I = m_{\text{rec}}R_e - bH. \end{aligned}$$

The recipient *alone* decrypts  $M_I = C_e - m_{\text{rec}}R_e$  and produces a plaintext bilateral receipt; membership of the decrypted  $M_I$  closes collusion. Full treatment and tests: *alberta-buck-notes-unilateral* (`unilateral_a2.py`, 11 Python + 7 Solidity).

## 5 B1 – The Public Issuer Names the Bearer Depositor

This is the dual, and it answers the standing question: *can a B1 issuer (public Identity) learn the unencrypted Identity of a depositor unknown at mint?* Yes – collusion-resistantly, and privately from Mallory.

### 5.1 The construction

At deposit the depositor  $D$  (identity  $m_{\text{dep}}$ , registered payout account  $(pk_{\text{dep}}, E_{\text{dep}} = (R_d, C_d))$ ) re-encrypts *its own* Identity under the public issuer’s key and publishes it in the **SpentB** event:

$$E_{\text{dep} \rightarrow \text{iss}} = (R_f, C_f) = (rG, M_{\text{dep}} + rpk_{\text{iss}}).$$

Only the issuer (holding  $sk_{\text{iss}}$ ) recovers  $M_{\text{dep}} = C_f - sk_{\text{iss}}R_f$ ; to Mallory it is IND-CPA noise. The depositor proves – on chain, hiding every Identity (**verifyDepositorBinding**) – knowledge of  $(m_{\text{dep}}, sk_{\text{dep}}, r)$ :

- |             |                                              |                                                                       |
|-------------|----------------------------------------------|-----------------------------------------------------------------------|
| <b>(E4)</b> | $pk_{\text{dep}} = sk_{\text{dep}}G$         | — the real payout-account key;                                        |
| <b>(E2)</b> | $C_d = m_{\text{dep}}G + sk_{\text{dep}}R_d$ | — the account is bound to $M_{\text{dep}} = m_{\text{dep}}G$ ;        |
| <b>(F1)</b> | $R_f = rG$                                   | — event-ciphertext randomness;                                        |
| <b>(F2)</b> | $C_f = m_{\text{dep}}G + rpk_{\text{iss}}$   | — $E_{\text{dep} \rightarrow \text{iss}}$ encrypts $M_{\text{dep}}$ . |

The *shared witness*  $m_{\text{dep}}$  couples (E2) and (F2): the Identity bound to the payout account is exactly the one encrypted for the issuer – so a colluding depositor cannot substitute a different (or junk) Identity. The membership half ( $\ell(M_{\text{dep}}) \in \text{rt}_{\text{id}}$ ) confirms it is genuine – though for B1 it is *also* implied, since the payout account is itself a registered record (the dual is even tighter than A2: the depositor is named, so the account *is* the membership witness).

### 5.2 The issuer-unilateral receipt

The issuer scans **SpentB**, decrypts  $E_{\text{dep} \rightarrow \text{iss}}$  with  $sk_{\text{iss}}$ , and produces a plaintext receipt naming *both* parties – *alone*, the mirror of the A2 recipient receipt. A **verifiable\_decrypt** proof (a Chaum-Pedersen DLEQ) makes it third-party-checkable: anyone confirms  $E_{\text{dep} \rightarrow \text{iss}}$  decrypts under  $pk_{\text{iss}}$  to  $M_{\text{dep}}$ , and  $M_{\text{dep}} \in \text{rt}_{\text{id}}$ . (Tested: **test\_issuer\_receipt\_names\_depositor**.)

### 5.3 Why it is collusion-resistant and private

**Theorem 3.** (B1 depositor naming.) *If **verifyDepositorBinding** accepts a B1 spend, the issuer recovers, from the event, a registered Identity  $M_{\text{dep}}$  the depositor controls; no colluding depositor can cash a B1 note while remaining un-nameable, and no third party learns  $M_{\text{dep}}$ .*

*Soundness:* (E2)+(F2) force  $E_{\text{dep} \rightarrow \text{iss}}$  to encrypt the *same*  $M_{\text{dep}}$  the registered payout account is bound to; a substituted ciphertext breaks (F2) (tested: **test\_collusion\_substituted\_ciphertext\_rejected**, and on chain **test\_vector\_substitutedCiphertext\_rejected**). *Privacy:* the sigma is HVZK and  $E_{\text{dep} \rightarrow \text{iss}}$  is IND-CPA under  $pk_{\text{iss}}$ , so Mallory cannot read  $M_{\text{dep}}$  nor link the depositor’s account to the issuer’s mint (tested: **test\_mallory\_cannot\_decrypt**, **test\_two\_deposits\_unlinkable**).

## 6 The Unified Spend Relation

One spend circuit, parameterised by flavour, replaces the divergent `spend_a.circom` (CP-pinned) and `spend.circom` (bearer):

$$\begin{array}{ll}
(\text{opening}) & cm = \text{Poseidon}(\text{flavor}, v, \rho, idHash, pred), \quad cm \in noteRoot; \\
(\text{identity}) & M = mG, \quad \ell(M) \in \mathbf{rt}_{id} \text{ (payout account bound to } M\text{)}; \\
(\text{flavor}) & \left\{ \begin{array}{l} \mathbf{A2:} \quad M_I = C_e - m R_e \text{ (decrypt issuer)} \\ \mathbf{A1:} \quad m = m_{\text{rec}} \text{ matches the addressed identity; } M_I \text{ plaintext} \\ \mathbf{B1:} \quad \text{know } \rho; \quad E_{\text{dep} \rightarrow \text{iss}} = (rG, MG + r pk_{\text{iss}}) \end{array} \right. \\
(\text{nullifier}) & nf = \text{Poseidon}(\rho, idHash, \text{tag}).
\end{array}$$

The *identity* and *flavor* lines are the one gadget in its three forms. The EC parts are EIP-196 sigmas (`verifyDepositCoupling` for A2, `verifyDepositorBinding` for B1, both shipped); the membership and the opening are the native Poseidon-Merkle circuit pieces.

## 7 What This Simplifies and Strengthens

| Aspect                      | Account-centric (today)                         | Identity-axis (this design)                                                  |
|-----------------------------|-------------------------------------------------|------------------------------------------------------------------------------|
| Spend circuits              | two (CP-pinned A-spend; bearer B-spend)         | one, parameterised by the flavor line                                        |
| "M is registered"           | implicit per-account; PS-credential or CP trick | one native membership in $\mathbf{rt}_d$                                     |
| Issuer binding              | Schnorr (public) / blinded-CP (A2, account key) | the one gadget (A2 toward $M_{\text{rec}}$ , B1 toward $pk_{\text{iss}}$ )   |
| B1 issuer-names-depositor   | optional, "honest depositor" only               | <b>enforced, collusion-resistant</b> ( <code>verifyDepositorBinding</code> ) |
| Key loss on addressed notes | terminal (note bound to mint-time $sk$ )        | recoverable (re-register a fresh account on $M$ )                            |
| Account in the design       | load-bearing                                    | interchangeable envelope matching $M$                                        |

## 8 Key-Management Semantics

Moving the authority to Identity  $M$  changes what a spend *requires*: knowledge of the Identity scalar  $m = H(\text{id})$  plus a KYC-registered account bound to  $M$ . Two consequences, stated honestly:

- **Recovery, not finality.** An addressed note survives the loss of any single account's key: re-register a fresh account on the same  $M$  (the KYC issuer re-certifies the real person) and deposit. The current account-pinned model treats loss as terminal; the identity-axis makes it recoverable, with the KYC issuer as the recovery anchor. Both can coexist (offer an account-pinned opt-in for self-custody finality).
- **No PII-grind shortcut.** Although  $m = H(\text{id})$  is recoverable from  $\text{id}$ , guessing it does *not* enable theft: spending also requires a *registered* account bound to  $M$  (constraint E2/E4), and an attacker cannot make the KYC issuer certify someone else's  $M$ . The non-grindable-identities assumption ( $H$  a random oracle) and the KYC gate together close the shortcut.

## 9 Implementation Status

Built and tested (both EVM-verified):

- **A2 deposit coupling** – `alberta_buck/wallet/unilateral_a2.py`, `IdentityRegistry.verifyDepositCoupling`; `test_unilateral_a2.py` (11) + `UnilateralA2.t.sol` (7).
- **B1 depositor binding** (this document’s dual) – `alberta_buck/wallet/b1_binding.py`, `IdentityRegistry.verifyDepositorBinding`; `test_b1_binding.py` (8) + `B1Binding.t.sol` (6).
- The shared `IdentityTree` accumulator and the two unilateral receipts (recipient names issuer; issuer names depositor).
- **The unified membership circuit** – `circuits/identity_membership.circom`: the native Poseidon-Merkle proof that the counterparty Identity  $M = (M_x, M_y)$  is a member of  $\text{rt}_{\text{id}}$ , hashing *byte-for-byte* with `IdentityTree`. Compiled ( $\sim 5.7\text{K R1CS}$ ), wired through a Groth16 setup (reusing the dev `pot13` ptau) to the exported `IdentityMembershipVerifier`, and on-chain-verified: a real proof of a Python-tree member verifies ( $\sim 271\text{K gas}$ , constant), a tampered root is rejected.

Test ledger.

| Suite                                    | Count    | Demonstrates                                                                                           |
|------------------------------------------|----------|--------------------------------------------------------------------------------------------------------|
| <code>test_unilateral_a2.py</code>       | 11       | A2 mint/coupling/receipt; collusion un-nameable + un-spendable                                         |
| <code>test_b1_binding.py</code>          | 8        | B1 binding; issuer-unilateral receipt names depositor; collusion-substitution rejected; Mallory blind; |
| <code>UnilateralA2.t.sol</code>          | 7        | on-chain <code>verifyDepositCoupling</code> parity + soundness                                         |
| <code>B1Binding.t.sol</code>             | 6        | on-chain <code>verifyDepositorBinding</code> parity + soundness                                        |
| <code>IdentityMembership.t.sol</code>    | 2        | on-chain Groth16 membership proof (Python-tree member) verifies; tampered root rejected                |
| <code>test_identity_membership.sh</code> | (script) | the circuit accepts a genuine member, rejects a wrong root (witness-level)                             |

**Status:** all 34 `forge/pytest` tests pass, no regression in the adjacent identity / A2 suites.

**Remaining (one relation):** only the *non-native* tie – proving that the circuit’s witnessed  $(M_x, M_y)$  is the very point committed in the on-chain EIP-196 sigma ( $P_I = M + bH$  for A2; the *eIss* plaintext for B1). That is one emulated- $\mathbb{G}_1$  relation bolted onto the (now built) native membership; it is proved off chain on the Holochain agent that holds the tree and verified on chain in constant gas. Plus the accumulator-root maintenance on registration (the *commit-before-use* discipline).

### 9.1 Migration

This supersedes the account-pinned `spend_a` CP path and the in-circuit-less B-spend with the unified relation, rotated in via the governance verifier slots (`setMintVerifier` / `setA2MintVerifier`). The two EIP-196 bindings shipped here are forward-compatible: they are the EC halves of the unified spend, usable on-chain today and inside the unified circuit tomorrow.

### 9.2 Cross-references

- *alberta-buck-notes-unilateral* – the A2 corner, in full.
- *alberta-buck-notes-decryptability* – the gap this closes and the "identity-targeted alternative" that this generalises to all flavours.

- *alberta-buck-identity* – the Fountain, registration, and the ElGamal / Chaum-Pedersen primitives reused throughout.